

Livet som sys.admin - Linux och bash-script

Övningshäfte

Dator- och kommunikationsteknik nivå 2 & Tillämpad programmering EE25
Storsjögymnasiet

Gjord av Marcus Weiderstål

Innehållsförteckning

Om det här häftet3

Kom igång – installera din virtuella maskin3

Del 1 – Kom igång med terminalen5

Del 2 – Filer och mappar7

Del 3 – Rättigheter för filer och mappar8

Del 4 – Pakethantering med apt10

Del 5 – Sätta upp en webbserver med Apache11

Del 6 – Fjärranslutning med SSH12

Del 7 – Administrera och hantera ett Linux-system14

Del 8 – Ubuntu Server: Linux utan grafiskt gränssnitt17

Bonus – sätt din prägel på terminalen19

Sammanfattning – checklista19

Fördjupande uppgifter21

Del 9 – Bashscripting för systemadministratörer22

Del 10 – Dual boot: installera Linux Mint vid sidan av Windows28

Om det här häftet

Det här häftet ger dig grunderna i att arbeta i Linux via terminalen. Du kommer att sätta upp en egen virtuell Linux-maskin, lära dig navigera i filsystemet, skapa och hantera filer och mappar, förstå och ändra behörigheter, installera program med pakethanteraren apt, sätta upp en egen webbserver med Apache, ansluta till din server på distans med SSH, och lära dig verktyg för att administrera och felsöka ett Linux-system i vardagen. Till sist tar häftet steget från att köra kommandon för hand till att skriva egna bash-skript, så att du kan automatisera återkommande sysadmin-uppgifter istället för att göra samma sak manuellt varje gång.

Arbeta dig igenom delarna i ordning – senare avsnitt bygger på det du lärt dig tidigare. Varje avsnitt innehåller korta genomgångar av kommandon följt av uppgifter markerade så här:

Exempel: Så här ser en uppgift ut. Lös uppgiften i terminalen och visa resultatet för din lärare eller anteckna svaret där det behövs.

I det här häftet utgår vi från att du installerar en virtuell maskin med Ubuntu 24.04, enligt guiden nedan. Det fungerar lika bra med Ubuntu Server, WSL eller en fysisk dator med Ubuntu/Debian installerat – kommandona är desamma.

Kom igång – installera din virtuella maskin

Innan du börjar med övningarna behöver du en Ubuntu-maskin att arbeta i. Här installerar du VirtualBox och skapar en virtuell maskin med Ubuntu 24.04 LTS – det gör att du kan köra Linux i ett eget fönster på din vanliga dator, utan att påverka något annat.

Vad är virtualisering?

Virtualisering innebär att man låter en dator simulera en eller flera andra datorer i mjukvara. Ett program som kallas hypervisor (i vårt fall VirtualBox) delar upp den fysiska datorns resurser – processor, minne och lagring – och låter en eller flera virtuella maskiner (VM) använda dem som om de vore egna, fristående datorer.

En virtuell maskin har ett eget operativsystem, egna program och eget filsystem, helt separat från den fysiska datorn den körs på (värden, eller host). Ur den virtuella maskinens perspektiv ser det ut som en vanlig dator – den vet inte att den egentligen delar hårdvara med andra.

Det här är precis samma teknik som används i stor skala på riktiga servrar och i molntjänster (t.ex. AWS, Azure eller Google Cloud): istället för att ha en fysisk server per uppgift kör man många virtuella maskiner på samma fysiska hårdvara, vilket sparar både pengar och energi. Att kunna skapa och hantera virtuella maskiner är därför en central färdighet inom nätverks- och systemadministration.

Fördelen för oss i det här häftet är att du kan experimentera fritt i din virtuella maskin – installera fel paket, ta bort filer av misstag eller till och med krascha systemet – utan att din vanliga dator påverkas. Går något snett kan du helt enkelt ta bort den virtuella maskinen och skapa en ny.

1. Ladda ner och installera VirtualBox

VirtualBox är programmet som kör den virtuella maskinen åt dig.

- Gå till [virtualbox.org](https://www.virtualbox.org) och ladda ner installationsfilen för ditt operativsystem (Windows/Mac/Linux).
- Kör installationsfilen och klicka dig igenom installationen med standardinställningarna.

2. Ladda ner Ubuntu 24.04

- Gå till ubuntu.com/download/desktop och ladda ner Ubuntu 24.04 LTS (Desktop-utgåvan). Det är den utgåvan guiden nedan utgår från – den har både grafiskt gränssnitt och terminal, vilket gör den enkel att komma igång med.
- Ladda ner .iso-filen. Det är en skivavbild som fungerar som installationsmedia för den virtuella maskinen.
- Notera var filen sparas (t.ex. mappen Hämtade filer) – du behöver hitta den igen strax.

3. Skapa en ny virtuell maskin

- Öppna VirtualBox och klicka på Ny (New).
- Ge maskinen ett namn, t.ex. Ubuntu24. VirtualBox brukar känna igen namnet och fylla i Typ: Linux och Version: Ubuntu (64-bit) automatiskt.
- Under ISO Image, välj den .iso-fil du laddade ner i steg 2 – det gör att den virtuella maskinen startar från Ubuntu's installationsskiva när du kör den.
- Tilldela minne (RAM): minst 4096 MB om din dator klarar det (Ubuntu's rekommendation för Desktop-utgåvan). Går det inte, fungerar det ofta med 2048 MB också, men lite långsammare.
- Skapa en virtuell hårddisk: välj VDI, Dynamiskt allokerad, minst 25 GB.
- Klicka igenom guiden till Slutför (Finish) för att skapa maskinen.

4. Installera Ubuntu

Starta den nya virtuella maskinen (klicka Start). Ubuntu's installationsguide startar automatiskt:

- Välj språk och tangentbordslayout (svenska fungerar bra).
- Välj Normal installation.
- Välj Radera disk och installera Ubuntu – detta gäller bara den virtuella hårddisken du just skapade, inte din riktiga dator.

- Skapa ett användarkonto: ange ditt namn, ett datornamn, användarnamn och lösenord. Skriv ner användarnamn och lösenord – du kommer att behöva dem för både inloggning, sudo och SSH senare i häftet.
- Låt installationen köra klart och starta om när du blir ombedd.

5. Efter installationen

Logga in med användarnamnet och lösenordet du skapade. Öppna sedan en terminal och uppdatera systemet direkt:

```
sudo apt update  
sudo apt upgrade
```

6. Nätverksinställningar (viktigt för SSH-delen)

Längre fram i häftet ska du kunna ansluta till din virtuella maskin med SSH, gärna från en annan dator. Då behöver den virtuella maskinen ett eget IP-nummer på samma nätverk som din dator, istället för att gömma sig bakom VirtualBox.

- Stäng av den virtuella maskinen.
- Markera maskinen i VirtualBox och gå till Inställningar (Settings) → Nätverk (Network).
- Ändra Ansluten till (Attached to) från NAT till Nätverksbryggat (Bridged Adapter), och välj ditt vanliga nätverkskort i listan.
- Starta maskinen igen. Kör `ip a` i terminalen för att se det nya IP-numret – det är detta du använder för att ansluta med SSH senare.

Uppgift 0.1: Installera VirtualBox, ladda ner Ubuntu 24.04 och skapa en virtuell maskin enligt guiden ovan. Se till att installationen går hela vägen till att du kan logga in.

Uppgift 0.2: Kör `sudo apt update` och `sudo apt upgrade` i din nya maskin. Kör därefter `ip a` och anteckna maskinens IP-adress – du kommer att behöva den senare i häftet.

Del 1 – Kom igång med terminalen

Vad är en terminal?

Terminalen (även kallad skal eller shell) är ett textbaserat gränssnitt där du styr datorn genom att skriva kommandon istället för att klicka i ett grafiskt gränssnitt. Det vanligaste skalet i Linux heter bash. Att kunna terminalen är grundläggande för allt arbete med servrar och nätverksutrustning.

Var hittar jag terminalen?

I Ubuntu Desktop finns det flera sätt att öppna en terminal:

- Snabbast: tryck Ctrl + Alt + T på tangentbordet.
- Klicka på Activities (eller tryck på Super-tangenten/Windows-tangenten) längst upp till vänster, skriv terminal och tryck Enter.
- Högerklicka på skrivbordet och välj Open Terminal, om alternativet finns.

Ett nytt terminalfönster med en prompt (t.ex. elev@ubuntu24:~\$) dyker upp – det är där du skriver alla kommandon i det här häftet. Du kan öppna flera terminalfönster samtidigt om du vill ha mer än ett igång parallellt.

Var du befinner dig – pwd och ls

Kommandot pwd (print working directory) visar vilken mapp du står i just nu:

```
pwd
```

Kommandot ls listar innehållet i mappen du står i:

```
ls
ls -l      # lång lista med rättigheter, storlek och datum
ls -a      # visar även dolda filer (som börjar med .)
ls -la     # kombination av båda
```

Att flytta sig – cd

Med cd (change directory) byter du mapp:

```
cd Dokument    # gå in i mappen Dokument
cd ..          # gå upp en nivå
cd ~           # gå till din hemkatalog
cd /           # gå till filsystemets rot
cd -           # gå tillbaka till föregående mapp
```

Skillnaden mellan absolut och relativ sökväg är viktig: en absolut sökväg börjar alltid med / (t.ex. /home/elev/Dokument), medan en relativ sökväg utgår från mappen du redan står i (t.ex. Dokument).

Linux mappstruktur – vad finns var?

Till skillnad från Windows, som har flera enhetsbokstäver (C:, D: ...), har Linux en enda mappstruktur som börjar i roten (/). Allt – hårddiskar, USB-minnen och nätverksenheter – monteras in någonstans under den strukturen. Här är de mappar du stöter på oftast:

- / – roten. Toppen av hela filsystemet, allt annat ligger under den.
- /home – hemkataloger för vanliga användare, t.ex. /home/elev. Dit hamnar du normalt när du loggar in.
- /root – root-användarens (administratörens) hemkatalog. Inte samma sak som / (roten).
- /etc – konfigurationsfiler för systemet och installerade program.

- /var – variabel data som ändras medan systemet körs, t.ex. loggfiler (/var/log) och webbserverns filer (/var/www).
- /bin och /usr/bin – körbara program och kommandon. Även kommandon som ls och cat är egentligen filer som ligger här.
- /tmp – tillfälliga filer. Innehållet raderas ofta vid omstart.
- /dev – enheter (hårddiskar, USB-minnen med mera), representerade som filer.
- /proc – en virtuell mappstruktur med information om systemet och pågående processer.

Du behöver inte memorera alla direkt, men det är bra att känna igen dem när du ser dem – särskilt /etc och /var, som du kommer att stöta på i resten av häftet (till exempel Apaches konfiguration i /etc/apache2 och webbroten i /var/www).

I Linux är allt en fil

En av grundidéerna i Unix och Linux är att nästan allt representeras som en fil – inte bara dokument och kod, utan även hårdvaruenheter, processer och information om själva systemet. Det gör att samma enkla verktyg (cat, ls och liknande) fungerar på nästan vad som helst i systemet, istället för att varje sak behöver ett eget specialprogram.

Några exempel:

- /dev/sda (eller liknande namn) representerar en hel hårddisk – som en fil.
- /proc/cpuinfo innehåller information om datorns processor, och kan läsas precis som vilken textfil som helst.
- /proc/meminfo visar aktuell minnesanvändning i realtid.

Testa själv:

```
cat /proc/cpuinfo | less      # bläddra i info om processorn (avsluta med q)
cat /proc/meminfo | less     # bläddra i info om minnesanvändningen
ls -l /dev | less            # se enheter representerade som filer
```

Lägg märke till att typstecknet längst till vänster i ls -l ibland är c eller b istället för - (vanlig fil) eller d (mapp) – det betyder att det är en enhetsfil. Det förklaras närmare i Del 3, som handlar om rättigheter för filer och mappar.

Övrigt smått och gott

```
clear      # rensar terminalfönstret
man ls     # visar manualen för ett kommando (avsluta med q)
history    # visar tidigare körda kommandon
```

Uppgift 1.1: Öppna en terminal. Ta reda på vilken mapp du står i med pwd, och lista innehållet med ls -la. Vilka dolda filer/mappar hittar du?

Uppgift 1.2: Navigera till din hemkatalog med cd ~. Gå därefter till filsystemets rot (/) och lista vad som finns där. Vilka mappar känner du igen?

Uppgift 1.3: Använd man för att ta reda på vad flaggan -h gör tillsammans med kommandot ls. Testa den.

Uppgift 1.4: Kör ls / och identifiera minst fyra av mapparna som beskrivs ovan. Vad tror du ligger i /usr? Använd gärna man hier eller sök på webben om du är osäker.

Uppgift 1.5: Kör cat /proc/cpuinfo och cat /proc/meminfo. Hur många processorkärnor (leta efter processor) listas, och hur mycket totalt minne (MemTotal) har din virtuella maskin?

Del 2 – Filer och mappar

Skapa filer och mappar

```
touch minfil.txt      # skapar en tom fil
mkdir minmapp         # skapar en mapp
mkdir -p a/b/c        # skapar flera nästlade mappar på en gång
```

Kopiera, flytta och byta namn

```
cp minfil.txt kopia.txt      # kopiera en fil
cp -r minmapp minmapp_kopia  # kopiera en mapp (rekursivt)
mv minfil.txt nyttnamn.txt   # byta namn (mv = move)
mv nyttnamn.txt minmapp/     # flytta filen till en mapp
```

Ta bort filer och mappar

```
rm minfil.txt          # ta bort en fil
rm -r minmapp          # ta bort en mapp och dess innehåll
rmdir tommapp          # ta bort en TOM mapp
```

Var försiktig med rm -r – det finns ingen papperskorg i terminalen. Filer som tas bort är borta.

Titta på filinnehåll

```
cat minfil.txt         # skriver ut hela filens innehåll
less minfil.txt        # bläddra i innehållet (avsluta med q)
```

Redigera filer med nano

nano är en enkel textredigerare som körs direkt i terminalen. Öppna en fil med nano filnamn – finns filen redan öppnas den, finns den inte skapas den (men sparas inte förrän du sparar den, se nedan):

```
nano minfil.txt
```

Du hamnar direkt i själva texten och kan skriva eller ändra som vanligt. Navigera med piltangenterna (inte musen). Längst ner i fönstret visas en rad kortkommandon, till exempel `^O` Write Out och `^X` Exit – `^` betyder Ctrl, så `^O` betyder Ctrl+O.

De kortkommandon du kommer använda mest:

```
Ctrl+O  # spara (Write Out) – bekräfta filnamnet med Enter
Ctrl+X  # avsluta nano – frågar om du vill spara ändringar först
Ctrl+W  # sök efter text i filen (Where is)
Ctrl+K  # klipp ut hela raden markören står på
Ctrl+U  # klistra in senast urklippta text
Ctrl+G  # visa hjälp med fler kortkommandon
```

Ett vanligt arbetsflöde: öppna filen med nano, skriv eller ändra texten, spara med Ctrl+O och bekräfta med Enter, och avsluta sedan med Ctrl+X. Försöker du avsluta med Ctrl+X utan att ha sparat frågar nano "Save modified buffer?" – svara Y (ja) för att spara, N (nej) för att kasta ändringarna, eller Ctrl+C för att ångra och stanna kvar i filen.

Uppgift 2.1: Skapa en mapp som heter `ovning2` och gå in i den. Skapa därefter tre tomma filer med touch: `fil1.txt`, `fil2.txt` och `fil3.txt`.

Uppgift 2.2: Öppna `fil1.txt` i nano och skriv in några rader valfri text. Spara med Ctrl+O och avsluta med Ctrl+X. Visa sedan innehållet med `cat`.

Uppgift 2.3: Öppna `fil1.txt` i nano igen. Lägg till en rad till, sök efter ett ord du skrev tidigare med Ctrl+W, och ta bort en hel rad med Ctrl+K. Spara och avsluta.

Uppgift 2.4: Kopiera `fil1.txt` till en ny fil som heter `fil1_kopia.txt`. Byt sedan namn på `fil2.txt` till `anteckningar.txt`.

Uppgift 2.5: Skapa mappstrukturen `projekt/kod/python` med ett enda kommando (tips: en flagga till `mkdir`).

Uppgift 2.6: Ta bort `fil3.txt`. Ta sedan bort hela mappen `ovning2` med dess innehåll.

Del 3 – Rättigheter för filer och mappar

Ägare, grupp och övriga

Varje fil och mapp i Linux har en ägare (user) och tillhör en grupp (group). Dessutom finns en tredje kategori: övriga (others), det vill säga alla andra användare på systemet. Rättigheter styrs separat för dessa tre: ägaren, gruppen och övriga.

Skapa en fil i din hemkatalog och kör `ls -l` på den för att se ägare, grupp och rättigheter:

```
touch minfil.txt
ls -l minfil.txt
-rw-r--r-- 1 elev elev 0 aug 6 10:00 minfil.txt
```

De två första kolumnerna efter rättighetssträngen visar ägare (elev) och grupp (elev) – troligen ditt eget användarnamn på båda ställena. Det beror på att Linux automatiskt skapar en grupp med samma namn som användaren när kontot skapas.

Jämför det här med en systemfil som redan finns, till exempel `/etc/passwd`, som istället ägs av root. Det är samma anledning till att du längre fram, i Del 5, kommer att behöva `sudo` för att redigera filer i Apaches webbbrot `/var/www/html` – de filerna ägs av root, inte av din egen användare.

Läsa rättighetssträngen

Strängen `-rw-r--r--` (eller `drwxr-xr-x` för mappar) består av fyra delar: första tecknet visar typ (`-` för fil, `d` för mapp), och därefter tre grupper om tre tecken vardera: rättigheter för ägare, grupp och övriga.

```
-  rw-  r--  r--
  |      |      |
  |      |      |  övriga:  r-- (endast läsa)
  |      |      |  grupp:   r-- (endast läsa)
  |      |      |  ägare:   rw- (läsa och skriva)
```

Varje position betyder:

- `r` (read) – läsa filens innehåll, eller lista en mapps innehåll
- `w` (write) – ändra filens innehåll, eller skapa/ta bort filer i en mapp
- `x` (execute) – köra filen som ett program/skript, eller gå in i en mapp med `cd`

Ett `-` istället för en bokstav betyder att den rättigheten saknas.

Kom ihåg enhetsfilerna i `/dev` från Del 1? Där kan det första tecknet istället vara `c` (character device) eller `b` (block device) – det är samma typkodning, bara fler typer än bara vanlig fil och mapp.

Ändra rättigheter med `chmod`

Med `chmod` (change mode) ändrar du rättigheter. Det vanligaste sättet är att ange dem som en tresiffrig oktal kod, där varje siffra är summan av `r=4`, `w=2` och `x=1` för respektive kategori (ägare, grupp, övriga):

```
chmod 644 minfil.txt      # ägare: rw-, grupp: r--, övriga: r-- (vanligt för filer)
chmod 755 script.sh       # ägare: rwx, grupp: r-x, övriga: r-x (vanligt för mappar/körbara filer)
chmod +x backup.sh        # lägg till körrättighet för alla
chmod u+w fil.txt         # lägg till skrivrättighet enbart för ägaren (u = user)
```

Bokstäverna `u`, `g`, `o` och `a` syftar på user (ägare), group, others och all (alla tre), och kombineras med `+` eller `-` för att lägga till eller ta bort en rättighet.

Ändra ägare och grupp med `chown` och `chgrp`

Med `chown` (change owner) byter du ägare och/eller grupp för en fil eller mapp:

```
sudo chown elev minfil.txt          # byt ägare till användaren elev
sudo chown elev:elev minfil.txt     # byt både ägare och grupp till
elev
sudo chown -R elev:elev minmapp     # gör samma sak rekursivt för en
hel mapp
sudo chgrp www-data minfil.txt      # byt enbart grupp till www-data
```

www-data är den grupp och användare som Apache normalt kör som (du sätter upp Apache i Del 5). Ett vanligt sätt att jobba är att låta din egen användare vara ägare (så att du kan redigera filerna direkt) och sätta gruppen till www-data, så att webbservern också kan komma åt filerna.

Uppgift 3.1: Skapa en fil i din hemkatalog, t.ex. med touch minfil.txt, och kör ls -l på den. Vem är ägare och vilken grupp tillhör filen?

Uppgift 3.2: Kör ls -l /etc/passwd. Vem äger filen, och vilka rättigheter har ägare, grupp och övriga? Vad tror du skulle hända om du försökte redigera filen med nano utan att använda sudo?

Uppgift 3.3: Skapa en fil hemlig.txt och ta bort all läsrättighet för grupp och övriga med chmod, så att endast du som ägare kan läsa filen. Kontrollera med ls -l att det blev rätt.

Uppgift 3.4: Skriv ett litet skalskript (en fil som börjar med #!/bin/bash) och försök köra det direkt med ./skript.sh. Vad händer om du inte har gett filen körrättighet? Ge den rättigheten med chmod och testa igen.

Uppgift 3.5: Skapa en mapp och en fil i den, ändra sedan ägare och grupp på mappen till www-data med chown -R. Vad innebär det praktiskt för vem som kan läsa/skriva filerna nu?

Del 4 – Pakethantering med apt

Vad är apt?

Du körde faktiskt redan sudo apt update och sudo apt upgrade i installationsguiden i början av häftet – nu ska vi titta närmare på vad de kommandona egentligen gör.

apt (Advanced Package Tool) är pakethanteraren i Ubuntu/Debian. Den laddar ner och installerar program (paket) från förvalda paketförråd (repositories) och håller reda på beroenden mellan paket, så att du slipper leta upp och installera allt manuellt.

sudo – att köra kommandon som administratör

De flesta åtgärder som ändrar systemet (installation, uppdateringar, konfiguration av tjänster) kräver administratörsrättigheter. Det gör man genom att skriva sudo före kommandot. Du får då ange ditt lösenord.

Uppdatera paketlistor och system

```
sudo apt update      # hämtar aktuell info om vilka paket/versioner  
som finns  
sudo apt upgrade     # uppgraderar installerade paket till senaste  
version
```

Kör alltid apt update innan du installerar något nytt, så att du får den senaste paketinformationen.

Installera och ta bort paket

```
sudo apt install <paketnamn>    # installera ett paket  
sudo apt remove <paketnamn>     # avinstallera ett paket (lämnar  
kvar inställningsfiler)  
sudo apt purge <paketnamn>      # avinstallera OCH ta bort  
inställningsfiler  
sudo apt autoremove             # städa bort paket som inte längre  
behövs
```

Söka och lista paket

```
apt search <sökord>              # sök efter paket  
apt show <paketnamn>            # visa detaljerad information om ett paket  
apt list --installed            # lista alla installerade paket
```

Uppgift 4.1: Kör sudo apt update. Vad händer i terminalen? Beskriv med egna ord vad kommandot faktiskt gjorde.

Uppgift 4.2: Sök efter paketet tree med apt search. Installera det sedan och testa att köra kommandot tree i en mapp med filer.

Uppgift 4.3: Ta reda på hur många paket som är installerade på ditt system just nu med apt list --installed (tips: kombinera med kommandot wc -l).

Uppgift 4.4: Avinstallera paketet tree igen med apt remove.

Del 5 – Sätta upp en webbservar med Apache

Installera Apache

Apache (apache2 i Ubuntu/Debian) är en av världens mest använda webbservrar. Installera den med apt:

```
sudo apt update  
sudo apt install apache2
```

Starta, stoppa och kontrollera status

Apache körs som en tjänst (service) som hanteras av systemd:

```
sudo systemctl status apache2    # visa status för tjänsten
sudo systemctl start apache2     # starta tjänsten
sudo systemctl stop apache2      # stoppa tjänsten
sudo systemctl restart apache2   # starta om tjänsten
sudo systemctl enable apache2    # gör så att Apache startar
                                  automatiskt vid uppstart
```

Testa att servern fungerar

Öppna en webbläsare och gå till `http://localhost` (om du testar på samma maskin) eller till serverns IP-adress, t.ex. `http://192.168.1.50`. Du bör se Apaches standardsida "Apache2 Ubuntu Default Page".

Vill du ta reda på serverns IP-adress kan du använda:

```
ip a
```

Webbrotens plats

De filer som Apache visar på webben ligger som standard i mappen `/var/www/html/`. Det är den mappen du behöver redigera för att ändra vad webbsidan visar.

```
cd /var/www/html
ls -l
sudo nano index.html    # redigera standardsidan (kräver sudo
                          eftersom mappen ägs av root)
```

Brandvägg (om ufw är aktiverat)

Om du har brandväggen ufw aktiv på servern behöver du tillåta webbtrafik:

```
sudo ufw allow "Apache"
sudo ufw status
```

Uppgift 5.1: Installera apache2 och kontrollera att tjänsten är igång med systemctl status. Besök sedan `http://localhost` i en webbläsare på servern och ta en skärmbild av standardsidan.

Uppgift 5.2: Ta reda på din servers IP-adress med `ip a`. Om du har en annan dator på samma nätverk, testa att besöka servern därifrån via IP-adressen.

Uppgift 5.3: Gå till `/var/www/html` och gör en säkerhetskopia av `index.html` (döp kopian till `index_original.html`). Redigera sedan `index.html` med nano så att sidan istället visar ditt namn och rubriken "Min första webbserver". Ladda om sidan i webbläsaren och kontrollera att ändringen syns.

Uppgift 5.4 – utmaning: Skapa en ny fil `om.html` i `/var/www/html` med lite valfri text. Besök sedan `http://localhost/om.html` i webbläsaren. Fungerade det direkt? Förklara varför/varför inte utifrån hur webbservern hittar filer.

Del 6 – Fjärranslutning med SSH

Vad är SSH?

SSH (Secure Shell) är ett protokoll som låter dig logga in på och styra en annan dator över nätverket, via en krypterad förbindelse. Det är så man i praktiken sköter servrar och nätverksutrustning i verkligheten – man sitter sällan framför själva servern, utan ansluter till den på distans från sin egen dator.

SSH består av två delar: en klient (det program du kör på din egen dator för att ansluta) och en server (tjänsten som körs på maskinen du vill ansluta till och som tar emot inloggningar). All trafik krypteras, till skillnad från äldre protokoll som telnet där allt, inklusive lösenord, skickas i klartext.

Installera SSH-server

På den dator/server du vill kunna ansluta till installerar du paketet openssh-server:

```
sudo apt update  
sudo apt install openssh-server
```

Kontrollera att tjänsten är igång:

```
sudo systemctl status ssh
```

Precis som med Apache kan du starta, stoppa och aktivera tjänsten vid uppstart:

```
sudo systemctl start ssh  
sudo systemctl enable ssh
```

Tillåt lösenordsinloggning

Som standard går det bra att logga in med lösenord via SSH, men det är värt att dubbelkolla – särskilt om du installerade OpenSSH direkt i Ubuntu Server-installationen (Del 8), där det ibland är förvalt till enbart nyckelinloggning. Öppna konfigurationsfilen:

```
sudo nano /etc/ssh/sshd_config
```

Leta upp raden med PasswordAuthentication och se till att den ser ut så här (ta bort ett eventuellt # framför raden om den är utkommenterad):

```
PasswordAuthentication yes
```

Spara, stäng, och starta om tjänsten så att ändringen börjar gälla:

```
sudo systemctl restart ssh
```

Ta reda på serverns IP-adress

För att kunna ansluta behöver du veta serverns IP-adress:

```
ip a
```

Ansluta som klient

De flesta Linux- och Mac-datorer har en SSH-klient inbyggd i terminalen. I Windows finns SSH numera inbyggd i både Kommandotolken och PowerShell. Anslut med:

```
ssh användarnamn@ip-adress  
# exempel:  
ssh elev@192.168.1.50
```

Första gången du ansluter till en server får du en fråga om du litar på serverns "fingerprint" (identitet) – svara yes. Du blir sedan ombedd att ange lösenordet för användaren på servern.

Vill du avsluta SSH-sessionen och komma tillbaka till din egen dator:

```
exit
```

Brandvägg (om ufw är aktiverat)

Precis som för Apache kan SSH-trafik behöva tillåtas explicit i brandväggen:

```
sudo ufw allow ssh  
sudo ufw status
```

Uppgift 6.1: Installera openssh-server på din Linux-server och kontrollera att tjänsten är igång med `systemctl status ssh`.

Uppgift 6.2: Ta reda på serverns IP-adress med `ip a`.

Uppgift 6.3: Från en annan dator (eller en annan terminal om du kör en virtuell maskin), ssh:a in till din server med `ssh användarnamn@ip-adress`. Bekräfta fingerprinten och logga in med lösenordet.

Uppgift 6.4: När du är inloggad via SSH, kör `pwd` och `ls` för att visa att du faktiskt befinner dig på serverns filsystem och inte din egen dator. Avsluta sedan sessionen med `exit`.

Uppgift 6.5 – utmaning: Fundera på (eller undersök) varför det anses säkrare att ansluta med en SSH-nyckel istället för lösenord. Vad heter kommandot som genererar ett nyckelpar?

Del 7 – Administrera och hantera ett Linux-system

De sista bitarna i det här häftet är sådant du kommer att använda om och om igen när du jobbar med Linux i praktiken – oavsett om du administrerar servrar, felsöker nätverk eller bara vill jobba snabbare i terminalen.

Kombinera kommandon: pipes och omdirigering

Du har redan använt tecknet | (pipe) några gånger tidigare i häftet, till exempel `cat /proc/cpuinfo | less`, utan att det förklarats närmare. Varje kommando i terminalen tar normalt emot indata från tangentbordet och skickar sin utdata till skärmen. Pipe kopplar istället ihop två kommandon, så att utdata från det första blir indata till det andra – ungefär som ett löpande band där varje kommando gör ett steg i bearbetningen innan resultatet skickas vidare.

Det gör att du kan bygga kraftfulla verktyg genom att kombinera flera enkla kommandon, istället för att behöva ett stort program som gör allt på en gång:

```
cat /proc/cpuinfo | less      # skicka utdata från cat till less, så
du kan bläddra
ls -la | grep .txt           # visa bara de rader från ls som
innehåller .txt
apt list --installed | wc -l  # räkna antal rader (paket) som apt
list skriver ut
```

`grep` är ett av de mest användbara kommandona att kombinera med pipe. Det söker igenom text, rad för rad, och visar bara de rader som matchar det du letar efter – oavsett om texten kommer från en fil eller från ett annat kommando via pipe:

```
grep root /etc/passwd        # visa rader i filen som innehåller "root"
grep -i hej minfil.txt       # -i = skiljer inte på versaler/gemener
(Hej, hej, HEJ ...)
grep -r TODO ~/projekt      # -r = sök rekursivt genom alla filer i en
mapp och dess undermappar
```

Ibland vill du spara utdata från ett kommando i en fil istället för att visa den i terminalen. Det gör du med `>` (skriv över) och `>>` (lägg till). Tänk på att `>` alltid skriver över hela filens tidigare innehåll, medan `>>` bygger vidare på det som redan finns:

```
echo "hej" > fil.txt          # skriv till en fil (skriver över
befintligt innehåll)
echo "igen" >> fil.txt         # lägg till en rad i slutet av filen, utan
att skriva över
ls -la > lista.txt            # spara hela utdata från ett kommando i en
fil
```

Nätverkskommandon

Eftersom du läser en nätverksinriktad kurs är de här kommandona extra viktiga – de är ofta det första du testar när du ska felsöka varför något inte kan nå varandra över nätverket.

`ping` skickar ett litet testmeddelande till en annan dator och väntar på svar. Det är oftast det första steget för att kontrollera om en dator eller server överhuvudtaget går att nå över nätverket:

```
ping -c 4 google.com         # skicka 4 pingar och avsluta, för att testa
om en värd svarar
```

`curl` och `wget` hämtar innehåll från en webbadress, precis som en webbläsare skulle göra – fast utan grafiskt gränssnitt. `curl` visar oftast innehållet direkt, medan `wget` är gjort för att spara ner filer:

```
curl http://localhost          # hämta och visa en webbsidas innehåll  
direkt i terminalen  
wget http://exempel.se/fil.zip # ladda ner en fil till din nuvarande  
mapp
```

ss visar vilka portar på din dator som lyssnar efter anslutningar just nu, och vilket program som lyssnar på var och en. Det är användbart för att kontrollera att t.ex. Apache (port 80) eller SSH (port 22) verkligen är igång och lyssnar:

```
ss -tulpn          # visa vilka portar som lyssnar (tcp/udp) och vilka  
program som använder dem
```

dig och nslookup slår upp DNS-information för ett domännamn, det vill säga vilken IP-adress som hör till t.ex. example.com. dig är det modernare och mer detaljerade av de två:

```
dig example.com      # slå upp DNS-information för en domän  
nslookup example.com # äldre, enklare alternativ till dig
```

Processer

Varje program som körs på datorn – från terminalen själv till Apache och SSH-tjänsten – körs som en eller flera processer. Varje process har ett eget unikt nummer, ett PID (Process ID), som du använder för att peka ut exakt vilken process du menar.

Med ps ser du vilka processer som är igång just nu. Kombinerat med grep kan du snabbt hitta en specifik process bland alla de andra:

```
ps aux              # lista alla körande processer, med PID,  
användare m.m.  
ps aux | grep apache2 # leta efter processer som har med apache2 att  
göra
```

Med kill kan du be en process att avsluta, genom att ange dess PID. Vanlig kill ber processen att avsluta snyggt (den kan hinna spara och städa upp); kill -9 tvingar istället fram en omedelbar avslutning, och bör bara användas om processen inte svarar på det vanliga kill:

```
kill 1234           # be process med PID 1234 att avsluta  
kill -9 1234        # tvinga fram avslutning om den inte lyssnar
```

Ibland vill du starta ett kommando som tar lång tid utan att behöva vänta på att det blir klart innan du kan skriva något annat. Läger du till & sist i kommandot startar det i bakgrunden, och du får tillbaka terminalen direkt:

```
sleep 100 &         # kör kommandot i bakgrunden (& i slutet)  
jobs                # lista bakgrundsjobb i den här terminalen  
fg                  # ta tillbaka senaste bakgrundsjobbet till förgrunden
```

Har du redan ett kommando igång i förgrunden (dvs. det du normalt kör, som blockerar terminalen tills det är klart) kan du pausa det med tangentkombinationen Ctrl+Z, och sedan skicka det till bakgrunden med kommandot bg utan att behöva starta om det.

Loggar

När något inte fungerar som det ska är loggfiler nästan alltid första stället att leta – de berättar vad ett program faktiskt gjorde, och ofta varför något gick fel. Ju bättre du är på att läsa loggar, desto snabbare hittar du problemet.

tail -f visar de sista raderna i en fil, och fortsätter sedan att visa nya rader allteftersom de skrivs – perfekt för att se vad som händer i realtid, till exempel medan du testar din webbserver:

```
tail -f /var/log/apache2/access.log      # följ loggfilen live (avsluta med Ctrl+C)
```

Många tjänster som hanteras av systemd (samma system som styr sudo systemctl start/stop, som du använt för apache2 och ssh) skriver också sina loggar till en central journal istället för, eller utöver, vanliga loggfiler. Den journalen läser du med journalctl:

```
sudo journalctl -u apache2      # visa loggar för apache2-tjänsten
sudo journalctl -u ssh -f       # följ SSH-loggen live, -f fungerar som
tail -f
```

Användare och grupper

I Del 3 bytte du ägare och grupp på filer med chown, men förutsatte att användarna redan fanns. Här ska du istället skapa och hantera själva användarkontona och grupperna – något en systemadministratör gör löpande när nya personer ska ha tillgång till ett system.

adduser skapar ett helt nytt användarkonto, med egen hemkatalog och eget lösenord. passwd används för att sätta eller byta lösenord på ett konto som redan finns:

```
sudo adduser kalle      # skapa en ny användare (frågar efter lösenord m.m.)
sudo passwd kalle       # byt lösenord för en användare
```

Varje användare tillhör en eller flera grupper, precis som filer kan ägas av en grupp (kom ihåg www-data från Del 3). Med groups ser du vilka grupper en användare tillhör, och med usermod -aG lägger du till användaren i ytterligare en grupp utan att ta bort de grupper de redan har:

```
groups kalle            # visa vilka grupper kalle tillhör
sudo usermod -aG www-data kalle # lägg till kalle i gruppen www-data
```

Flaggan -a (append) är viktig – utan den skulle usermod -G istället ersätta alla kalles befintliga grupper med bara den nya.

Arkiv och komprimering

Att arkivera innebär att slå ihop flera filer och mappar till en enda fil, vilket gör dem enklare att flytta eller skicka som en helhet. Ofta komprimeras arkivet samtidigt, så att det tar mindre plats. Det här stöter du på ofta – vid backuper (jämför med skriptet du kan bygga i de fördjupande uppgifterna), och när du laddar ner mjukvara som distribueras som ett arkiv.

tar är standardverktyget i Linux. Bokstäverna i flaggorna beskriver vad som ska göras: c (create) skapar ett nytt arkiv, x (extract) packar upp ett befintligt, z komprimerar/packar upp med gzip, v (verbose) visar vilka filer som hanteras, och f anger vilket filnamn som ska användas:

```
tar -czvf arkiv.tar.gz minmapp/ # packa en mapp till en komprimerad
arkiv-fil
tar -xzvf arkiv.tar.gz          # packa upp arkivet igen
```

zip och unzip fungerar på liknande sätt, men skapar .zip-filer istället – ett format som är särskilt vanligt om filerna ska öppnas på Windows eller Mac:

```
zip -r arkiv.zip minmapp/      # skapa en zip-fil istället
unzip arkiv.zip                # packa upp en zip-fil
```

Uppgift 7.1: Skapa några filer med olika ändelser i en mapp (t.ex. .txt och .sh). Använd ls -la tillsammans med grep för att lista bara filerna som slutar på .txt.

Uppgift 7.2: Kör ping -c 4 mot en webbplats du väljer själv. Testa sedan curl http://localhost mot din egen Apache-server och jämför vad du får ut av de två kommandona.

Uppgift 7.3: Starta ett kommando i bakgrunden, t.ex. sleep 300 &. Lista bakgrundsjobb med jobs, ta reda på processens PID med ps aux, och avsluta den med kill innan tiden gått ut.

Uppgift 7.4: Kör tail -f /var/log/apache2/access.log i en terminal. Besök sedan din webbserver i en webbläsare och se vad som händer i loggen. Avsluta med Ctrl+C när du är klar.

Uppgift 7.5: Skapa en ny användare med adduser, och lägg till den i gruppen www-data med usermod -aG. Kontrollera med groups att det blev rätt.

Uppgift 7.6: Packa en mapp med filer till en .tar.gz-fil. Packa sedan upp den igen i en ny mapp och kontrollera att innehållet stämmer.

Del 8 – Ubuntu Server: Linux utan grafiskt gränssnitt

Hittills har du jobbat i Ubuntu Desktop, som har ett grafiskt skrivbord med fönster och ikoner – du har bara valt att göra allt i terminalen. De allra flesta riktiga servrar kör istället Ubuntu Server, en utgåva som inte har något grafiskt gränssnitt alls. Allt sköts via terminalen, ofta på distans via SSH – precis det du lärde dig i Del 6.

I det här avsnittet installerar du en andra virtuell maskin, den här gången med Ubuntu Server, för att själv se skillnaden.

Varför Ubuntu Server?

Ett grafiskt gränssnitt tar upp minne, processorkraft och lagringsutrymme som en server hellre lägger på det den faktiskt ska göra – till exempel svara på webbförfrågningar eller hantera nätverkstrafik. Ubuntu Server är en betydligt mer nedbantad utgåva av samma Ubuntu, byggd för att köras utan skärm, mus eller tangentbord kopplat direkt till maskinen. Det är den typen av installation du kommer stöta på i princip varje gång du jobbar med riktiga servrar i arbetslivet.

Ladda ner Ubuntu Server 24.04

- Gå till ubuntu.com/download/server och ladda ner Ubuntu Server 24.04 LTS (.iso-filen).

Skapa en ny virtuell maskin

- Öppna VirtualBox och klicka på Ny (New), precis som tidigare.
- Ge maskinen ett eget namn, t.ex. UbuntuServer24, så du kan skilja den från din Desktop-maskin.
- Under ISO Image, välj .iso-filen du laddade ner i steget ovan.
- Tilldela minne (RAM): Server behöver betydligt mindre än Desktop – 1024–2048 MB räcker gott.
- Skapa en virtuell hårddisk: VDI, Dynamiskt allokerad. Server klarar sig med mindre utrymme än Desktop, t.ex. 10 GB.
- Klicka igenom guiden till Slutför (Finish).

Installera Ubuntu Server

Starta den nya maskinen. Första skillnaden du märker är att installationsprogrammet är textbaserat – du navigerar med piltangenterna och Tab, och väljer med mellanslag eller Enter, istället för att klicka med musen:

- Välj språk och tangentbordslayout.
- Nätverket ställs oftast in automatiskt (DHCP) – installationsprogrammet visar vilken IP-adress maskinen fått.
- Under Storage layout, välj att använda hela den virtuella hårddisken (motsvarar "Radera disk och installera" i Desktop-guiden).
- Fyll i Profile setup: ditt namn, ett servernamn, användarnamn och lösenord.
- Installationsprogrammet frågar om du vill installera OpenSSH-server direkt – kryssa i det. Då slipper du göra om Del 6 för att kunna ansluta till den här maskinen.
- Hoppa över Featured Server Snaps (extra program) om du inte vill installera något särskilt.
- Låt installationen bli klar och starta om när du blir ombedd.

Logga in – utan skrivbord

Efter omstarten möts du bara av en inloggningsprompt i textläge – inget skrivbord, inga fönster, inga ikoner. Logga in med användarnamnet och lösenordet du valde under installationen.

Testa några av kommandona du redan kan, t.ex. `pwd`, `ls` och `whoami`. De fungerar exakt likadant som i din Desktop-installation – det är samma Ubuntu-system under huven, bara utan det grafiska skalet ovanpå.

Uppgift 8.1: Installera Ubuntu Server 24.04 i en ny virtuell maskin enligt guiden ovan. Notera hur installationsprogrammet skiljer sig från Desktop-installationen du gjorde i början av häftet.

Uppgift 8.2: Logga in i din nya Server-installation. Kör `pwd`, `ls -la` och `whoami`. Fungerar de likadant som i din Desktop-maskin?

Uppgift 8.3: Ta reda på Server-maskinens IP-adress med `ip a`. Anslut till den med `ssh` från din Desktop-maskin (eller din vanliga dator), precis som du gjorde i Del 6.

Uppgift 8.4 – reflektion: Fundera på (eller diskutera med en klasskompis): varför har en riktig webbserver eller nätverksenhet i produktion sällan ett grafiskt gränssnitt installerat? Vilka fördelar ser du med Server-utgåvan jämfört med Desktop, och finns det några nackdelar?

Bonus – sätt din prägel på terminalen

Som en kul avslutning: du kan få terminalen att hälsa dig med ditt eget namn i stora bokstäver varje gång du öppnar den. Helt onödigt rent praktiskt – men ett bra sätt att se hur du kan anpassa din egen terminalmiljö.

Installera figlet

figlet är ett litet program som ritar text som stora bokstäver byggda av vanliga tecken, så kallad ASCII-konst. Installera det precis som vilket annat paket som helst:

```
sudo apt install figlet
```

Testa det direkt:

```
figlet Hej
```

.bashrc – din terminals startskript

Varje gång du öppnar ett nytt terminalfönster körs en fil som heter `.bashrc`, som ligger i din hemkatalog. Det är en dold fil (kom ihåg `ls -a` från Del 1) som du kan fylla med egna kommandon och inställningar som ska köras automatiskt varje gång du öppnar en ny terminal.

```
ls -a ~          # leta reda på .bashrc i listan
nano ~/.bashrc   # öppna filen i nano
```

Skrolla längst ner i filen och lägg till en rad som skriver ut ditt namn med figlet, till exempel:

```
figlet Elev
```

Spara (Ctrl+O) och stäng (Ctrl+X).

Starta om terminalen

Ändringar i `.bashrc` körs bara när ett nytt terminalfönster (eller en ny session) öppnas – inte i det fönster du redan har öppet. Stäng terminalen helt och öppna en ny, så bör ditt namn hälsa dig i stora bokstäver varje gång. Vill du se resultatet direkt utan att öppna ett nytt fönster kan du också köra:

```
source ~/.bashrc    # kör om .bashrc i det fönster du redan har öppet
```

Uppgift B.1: Installera figlet, lägg till en rad i din `.bashrc` som skriver ut ditt namn, och starta om terminalen (eller kör `source ~/.bashrc` för att se resultatet).

Sammanfattning – checklista

Bocka av allt eftersom du känner dig säker på momenten:

- ☐ Jag har installerat VirtualBox och en virtuell maskin med Ubuntu, och kan logga in i den.
- ☐ Jag kan navigera i filsystemet med `pwd`, `ls` och `cd`.
- ☐ Jag förstår skillnaden mellan absolut och relativ sökväg.
- ☐ Jag kan skapa, kopiera, flytta och ta bort filer och mappar.
- ☐ Jag kan redigera en textfil i terminalen med `nano`.
- ☐ Jag förstår vad `sudo` används till.
- ☐ Jag förstår vad ägare, grupp och övriga betyder för en fil.
- ☐ Jag kan läsa en rättighetssträng som `-rw-r--r--`.
- ☐ Jag kan ändra rättigheter med `chmod` och ägare/grupp med `chown`.
- ☐ Jag kan uppdatera, installera och avinstallera paket med `apt`.
- ☐ Jag kan installera Apache och kontrollera dess status med `systemctl`.
- ☐ Jag kan hitta och redigera filer i webbroten `/var/www/html`.
- ☐ Jag har testat att besöka min egen webbserver i en webbläsare.
- ☐ Jag förstår vad SSH är och varför det används.
- ☐ Jag kan installera och starta en SSH-server.
- ☐ Jag kan ansluta till en Linux-server med `ssh` från en annan dator.
- ☐ Jag kan kombinera kommandon med pipe (`|`) och söka text med `grep`.
- ☐ Jag kan skicka utdata till en fil med `>` och `>>`.

- ☐ Jag kan testa nätverk med ping, curl och ss.
- ☐ Jag kan lista och avsluta processer med ps och kill, och köra kommandon i bakgrunden.
- ☐ Jag kan följa loggar med tail -f och journalctl.
- ☐ Jag kan skapa en användare och lägga till den i en grupp.
- ☐ Jag kan packa och packa upp arkiv med tar.
- ☐ Jag har installerat Ubuntu Server och sett skillnaden mot Desktop-utgåvan.
- ☐ Jag har gjort terminalen lite mer min egen med figlet (bonus).

Fördjupande uppgifter

Är du klar i god tid? Här är några lite klurigare uppgifter som knyter ihop flera moment från häftet. De är inte ordnade efter svårighetsgrad – välj det som verkar mest intressant.

Guiderna ger dig en riktning, men du kommer troligen behöva söka upp lite information själv på internet eller i manualsidorna (man) för att lösa uppgifterna fullt ut. Det är en del av övningen – att lära sig hitta svar på egen hand är en viktig färdighet inom IT.

Fördjupande uppgift 1 – Skalskript: Skapa ett skalskript (backup.sh) som kopierar hela /var/www/html till en backupmapp med dagens datum i namnet, t.ex. backup_2026-08-06. Använd nano för att skriva skriptet, gör det körbart med chmod +x och kör det. Tips: kommandot date +%F ger dagens datum i formatet ÅÅÅÅ-MM-DD.

Guide:

1. Skapa filen med nano backup.sh.
2. Låt första raden vara #!/bin/bash – den talar om att filen ska köras med bash.
3. Spara dagens datum i en variabel: DATUM=\$(date +%F).
4. Kopiera mappen till en ny plats med det datumet i namnet: cp -r /var/www/html /var/www/backup_\${DATUM}.
5. Spara filen och gör den körbar: chmod +x backup.sh.
6. Kör skriptet med ./backup.sh och kontrollera med ls /var/www att backupmappen skapades.

Fördjupande uppgift 2 – Filöverföring med scp: Skapa en textfil på din egen dator. Använd scp (som bygger på SSH) för att kopiera filen till din Linux-server, och sedan en annan fil tillbaka från servern till din dator. Ta reda på själv hur scp-syntaxen ser ut, t.ex. med man scp.

Guide:

1. Skapa en textfil lokalt, t.ex. test.txt.
2. Skicka den till servern: scp test.txt användarnamn@ip-adress:/home/användarnamn/.
3. Ange lösenordet när du blir tillfrågad, precis som vid en vanlig SSH-inloggning.
4. Logga in på servern med ssh och kontrollera med ls att filen kom fram.
5. Testa att hämta en fil åt andra hållet: scp användarnamn@ip-adress:/sökväg/till/fil.txt . (punkten betyder "till nuvarande mapp").

Fördjupande uppgift 3 – Egna webbsidor och virtual hosts: Apache kan visa flera olika sidor på samma server. Börja med det enklaste sättet: skapa två mappar i webbroten, t.ex. map1 och map2, med varsin egen index.html, och besök dem via http://localhost/map1 respektive http://localhost/map2. Gå sedan vidare och testa det mer avancerade sättet: en helt egen webbplats med hjälp av en så kallad virtual host, så att sajten nås utan någon extra mapp i webbadressen.

Guide:

1. Skapa två mappar i webbroten: sudo mkdir /var/www/html/map1 /var/www/html/map2.
2. Skapa en enkel sida i var och en av dem, t.ex. sudo nano /var/www/html/map1/index.html (upprepa för map2).

3. Besök `http://localhost/map1` och `http://localhost/map2` i webbläsaren – Apache hittar undermappar i webbroten automatiskt, ingen extra konfiguration behövs.
4. Vill du gå vidare: skapa istället en helt fristående mapp, t.ex. `sudo mkdir /var/www/minsajt`, med en egen `index.html`.
5. Kopiera standardkonfigurationen som mall: `sudo cp /etc/apache2/sites-available/000-default.conf /etc/apache2/sites-available/minsajt.conf`.
6. Öppna `minsajt.conf` med `nano` och ändra `DocumentRoot` till `/var/www/minsajt` samt lägg till ett eget `ServerName`.
7. Aktivera den nya sajten: `sudo a2ensite minsajt.conf`.
8. Ladda om Apache: `sudo systemctl reload apache2`.
9. Testa t.ex. med `curl -H "Host: dittnamn" http://localhost` eftersom du saknar riktig DNS-post för namnet.

Fördjupande uppgift 4 – SSH-nycklar: Generera ett SSH-nyckelpar på din egen dator (eller i en andra terminal i samma virtuella maskin, om din dator saknar en SSH-klient) med `ssh-keygen`. Kopiera den publika nyckeln till servern med `ssh-copy-id`. Testa att logga in igen – du ska nu kunna ansluta utan att skriva lösenord. Förklara med egna ord vad som skiljer den publika och den privata nyckeln åt.

Guide:

1. Kör `ssh-keygen` på din egen dator och tryck `Enter` genom alla frågor för standardval.
2. Det skapas två filer i mappen `~/.ssh/`: `id_rsa` (privat, ska aldrig delas) och `id_rsa.pub` (publik, kan delas fritt).
3. Kopiera den publika nyckeln till servern: `ssh-copy-id användarnamn@ip-adress`.
4. Ange lösenordet en sista gång när du blir ombedd.
5. Testa att ansluta igen med `ssh användarnamn@ip-adress` – nu ska det inte krävas något lösenord.

Fördjupande uppgift 5 – Övervaka processer och resurser: Undersök kommandona `top` (eller `htop`, installeras via `apt`), `df -h` och `free -h`. Skriv ner vad varje kommando visar, och vilken information som skulle vara mest användbar om servern kändes trög.

Guide:

1. Kör `top` och titta på kolumnerna `%CPU` och `%MEM` för de olika processerna. Avsluta med `q`.
2. Installera en mer lättläst variant: `sudo apt install htop`, och kör `htop`.
3. Kör `df -h` för att se hur mycket diskutrymme som är ledigt/upptaget, angivet i G/M istället för byte.
4. Kör `free -h` för att se hur mycket minne (RAM) som används och hur mycket som är ledigt.
5. Anteckna vilket av kommandona du skulle titta på först om servern kändes trög, och varför.

Fördjupande uppgift 6 – Automatisera med cron: Använd `crontab -e` för att schemalägga skriptet du skrev i Fördjupande uppgift 1 så att det körs automatiskt varje natt klockan 03:00. Ta reda på hur cron-syntaxen (fem fält för tid) fungerar innan du skriver raden.

Guide:

1. Öppna din crontabell: `crontab -e` (välj `nano` om du blir tillfrågad om redigerare första gången).
2. De fem tidsfälten är: minut timme dag-i-månaden månad veckodag, i den ordningen.

3. Lägg till en rad längst ner: `0 3 * * * /full/sökväg/till/backup.sh` (kör 03:00 varje dag).
4. Spara och stäng filen – cron läser in ändringen automatiskt.
5. Kontrollera att jobbet finns med `crontab -l`.

Del 9 – Bashscripting för systemadministratörer

Nu när du kan navigera, hantera filer, sätta rättigheter och administrera ett Linux-system för hand är det dags att automatisera. Tänk dig en helt vanlig måndagsmorgon som sysadmin: du loggar in och ska kontrollera att gårdagens säkerhetskopior faktiskt kördes, att ingen av servrarna håller på att få slut på diskutrymme, och att natten inte gett upphov till en flodvåg av felmeddelanden i loggarna. Gör du det här för hand på tio servrar tar det en halvtimme – varje dag, året runt. Ett skript gör exakt samma kontroller på några sekunder, och kan dessutom köras automatiskt varje natt utan att någon behöver sitta uppe och vakta. Det är precis den typen av verktyg du bygger i den här delen: egna bash-skript som automatiserar återkommande sysadmin-uppgifter, istället för att göra samma sak manuellt varje gång. Delen är betydligt mer omfattande än de föregående – räkna med flera lektionstillfällen för att arbeta dig igenom den.

Skript, variabler och indata

Ett skript är helt enkelt en textfil med kommandon som körs i exakt den ordning de står, som om du skrivit dem i terminalen ett efter ett – fast automatiskt. Första raden i filen (shebang) talar om för Linux vilket program som ska tolka resten av innehållet:

```
#!/bin/bash
echo "Hej, världen!"
```

En vanlig textfil går inte att köra som ett program förrän den fått körrättighet. `chmod +x` gör precis det, och därefter kör du skriptet med `./` framför filnamnet – punkten och snedstreckat talar om att filen ligger i mappen du redan står i, istället för någonstans i `$PATH`:

```
chmod +x hej.sh
./hej.sh
```

En variabel skapas utan mellanslag runt likhetstecknet – `NAMN="värde"`, inte `NAMN = "värde"` – och läses genom att sätta ett dollartecken framför namnet. Precis som ett vanligt kommando kan ta emot argument på kommandoraden kan ditt eget skript göra samma sak: det första värdet som skickas in hamnar i `$1`, det andra i `$2` och så vidare, medan `$#` talar om hur många argument som faktiskt skickades in. Det gör att samma skript går att återanvända mot olika mappar, servrar eller användarnamn utan att du behöver ändra i koden varje gång:

```
NAMN="$1"
```

```
echo "Hej, $NAMN! Du gav mig $# argument."
```

Ibland vet du inte i förväg vad skriptet ska användas mot – då kan du istället fråga användaren direkt under körningen med `read`, som pausar skriptet tills någon skriver ett svar och trycker Enter:

```
read -p "Vad heter du? " NAMN
echo "Hej, $NAMN!"
```

Två vanor är värda att bygga in redan från början, eftersom de förebygger några av de vanligaste buggarna i bash-skript. Den ena är `${1:-standard}`, som ger en variabel ett standardvärde om inget argument angavs – utan den kraschar skriptet så fort någon glömmer ett argument. Den andra är att alltid citera variabler, som `"$@"` eller `"$1"`: utan citattecken delar bash upp värdet vid varje mellanslag, så att en sökväg som `"/home/min mapp"` plötsligt tolkas som två separata argument, `"/home/min"` och `"mapp"`, istället för ett:

```
GRANS="${1:-80}"
echo "Använder gränsvärde: $GRANS"
```

Uppgift 9.1: Skriv ett skript `sysinfo.sh` som tar emot ett valfritt namn som argument (annars används `hostname`), och skriver ut maskinens upptid (`uptime`), diskutrymme (`df -h`), minnesanvändning (`free -h`) och inloggade användare (`who`) – både till skärmen och till en loggfil med dagens datum i namnet.

Villkor och loopar

Ett `if`-block låter skriptet fatta beslut istället för att alltid göra samma sak – är villkoret sant körs den första grenen, annars hoppar skriptet vidare till `else`, ungefär som ett vägval:

```
if [ "$ALDER" -ge 18 ]; then
    echo "Myndig"
else
    echo "Inte myndig"
fi
```

En stor del av att skriva pålitliga sysadmin-skript handlar om att kontrollera saker innan du agerar på dem – annars riskerar skriptet att krascha mitt i, eller värre, göra något mot en fil eller mapp som inte ens finns. Filtester är därför bland de mest använda villkoren i den här typen av skript: `-e` kontrollerar att något överhuvudtaget existerar, `-f` att det specifikt är en fil, `-d` att det är en mapp, och `-r/-w/-x` om du har läs-, skriv- respektive körrättighet till det:

```
if [ ! -d "$MAPP" ]; then
    echo "Mappen finns inte" >&2
    exit 1
fi
```

Loopar låter skriptet upprepa samma kod för flera saker, istället för att du kopierar och klistrar in samma rader flera gånger. `for` går igenom en färdig lista – filer, argument eller en uppräkningslista – medan `while` fortsätter köra så länge ett villkor fortfarande är

sant, vilket passar bra t.ex. när du läser en fil rad för rad utan att veta i förväg hur många rader den innehåller:

```
for FIL in *.log; do
    echo "Hittade: $FIL"
done
```

Varje kommando du kör i Linux – inte bara dina egna skript – avslutar med en exit code lagrad i \$?, ett tal mellan 0 och 255. 0 betyder att allt gick som förväntat, vilket som helst annat tal betyder att något gick fel. Genom att kontrollera \$? efter ett viktigt kommando kan skriptet upptäcka och hantera fel istället för att blint fortsätta som om allt var bra. Det är samma grundidé som återkommer under namnet idempotens längre fram, om du går vidare till konfigurationshanteringsverktyg som Ansible: att alltid veta om något faktiskt lyckades innan du går vidare.

Uppgift 9.2: Skriv ett skript diskcheck.sh som går igenom filsystemen (df) och skriver ut en varning för de som använder mer än 80 % av utrymmet (eller ett gränsvärde du skickar in som argument). Lägg även till att den skriver ut "Det är lugnt" om 80% inte uppfylls.

Uppgift 9.3: Skriv ett skript logscan.sh som tar emot en mapp och ett gränsvärde i MB, och listar vilka .log-filer i mappen som är större än gränsvärdet.

Listor med arrayer

En array är en lista av flera värden lagrade i en enda variabel, istället för att du skapar VAR1, VAR2, VAR3 för hand. Det är särskilt användbart i sysadmin-sammanhang eftersom samma kontroll ofta ska göras mot flera saker på en gång – tre webbservrar, tio användare, en lista av tjänster som ska vara igång. Istället för att skriva samma kod flera gånger loopar du en enda gång över arrayen:

```
SERVRAR=( "webb1" "webb2" "db1" )

for VARD in "${SERVRAR[@]}"; do
    echo "Kontrollerar $VARD..."
done
```

`${SERVRAR[@]}` expanderar till samtliga element i arrayen, vilket är vad du loopar över, medan `${#SERVRAR[@]}` ger antalet element – praktiskt för att t.ex. skriva ut en sammanfattning som "3 av 3 servrar kontrollerade" när loopen är klar.

Uppgift 9.4: Skriv ett skript pingcheck.sh med en array av hostnames eller IP-adresser. Låt skriptet pinga varje värd (ping -c 1 -W 2 värdnamn) och skriva ut vilka som svarar och vilka som inte gör det. Avsluta med en sammanfattning: "X av Y värdar svarade".

Funktioner och felhantering

Ett skript som bara växer på längd blir snabbt svårt att läsa och underhålla – särskilt för någon annan som ska ta över det efter dig. En funktion samlar en avgränsad uppgift

under ett eget, beskrivande namn, vilket gör resten av skriptet lättare att följa och samma kod möjlig att återanvända utan att kopieras:

```
halsa() {  
    local NAMN="$1"  
    echo "Hej, $NAMN!"  
}  
  
halsa "Ada"
```

Ett skript som kraschar mitt i, med ett kryptiskt felmeddelande långt senare, är svårt att felsöka – särskilt om det körs automatiskt utan att någon tittar på. Kontrollera därför indata så tidigt som möjligt i skriptet och avsluta tydligt om något saknas, med ett läsbart felmeddelande och en exit code skild från 0:

```
if [ -z "$1" ]; then  
    echo "Fel: du måste ange ett argument" >&2  
    exit 1  
fi
```

Uppgift 9.5: Tidigare i häftet (Fördjupande uppgift 1) skrev du ett enkelt backupskript. Bygg nu en mer robust version, `backup.sh`, uppdelad i funktioner: `kontrollera_indata` (kontrollerar att källa och mål är angivna och att källan finns), `kontrollera_utrymme` (kontrollerar att det finns tillräckligt ledigt utrymme på målet), `kor_backup` (komprimerar källmappen till en tidsstämplad `.tar.gz`-fil) och `skriv_logg` (skriver alla meddelanden, med tidsstämpel, till både skärmen och en loggfil). Testa skriptet både när allt fungerar och när du anger en källmapp som inte finns.

Argument och flaggor med getopt

Jämför skillnaden mellan att behöva komma ihåg en exakt ordning på argumenten (`./skript.sh mapp 80 ja`) och att kunna skriva `./skript.sh -m mapp -g 80 -v` i valfri ordning, precis som riktiga Linux-kommandon fungerar. `getopts` läser flaggor på kommandoraden och låter dig bygga skript som känns igen och är mer förlåtande att använda:

```
while getopts "hvg:" FLAGGA; do  
    case $FLAGGA in  
        h) echo "Användning: $0 [-g gränsvärde] [-v] [-h]"; exit 0 ;;  
        v) VERBOSE=1 ;;  
        g) GRANS="$OPTARG" ;;  
        *) echo "Okänd flagga" >&2; exit 1 ;;  
    esac  
done
```

Ett kolon direkt efter en bokstav i flaggsträngen (som `g` i `"hvg:"` ovan) talar om att flaggan förväntar sig ett tillhörande värde, som hamnar i den inbyggda variabeln `$OPTARG`. En flagga utan kolon, som `h` och `v`, är istället bara ett på-eller-av-val utan eget värde.

Uppgift 9.6: Bygg om diskcheck.sh från uppgift 9.2 så att den tar emot gränsvärdet via flaggan -g istället för ett vanligt argument, och lägg till en -h-flagga som skriver ut en kort hjälptext och avslutar skriptet.

Felsökning av skript

Ibland gör ett skript inte alls vad du tänkt dig, utan att du förstår varför. Istället för att gissa och klistra in echo-rader överallt kan du köra hela skriptet med bash -x, som skriver ut varje rad – med variablernas faktiska värden isatta – i samma ögonblick den körs. Det gör det betydligt lättare att se exakt var något går snett:

```
bash -x backup.sh /hem/data /hem/backup
```

set -euo pipefail högst upp i skriptet ger dig tre skyddsnät på en gång, och är något erfarna sysadmins ofta lägger till som standard. -e stoppar skriptet direkt om något kommando misslyckas, istället för att skriptet blint fortsätter som om inget hänt. -u stoppar skriptet om du råkar använda en variabel som aldrig satts – ofta ett tecken på en felstavning som annars är svår att upptäcka. pipefail ser till att ett fel tidigt i en kedja av kommandon (|) faktiskt räknas som ett fel, istället för att bara det sista kommandots resultat räknas:

```
#!/bin/bash
set -euo pipefail
```

Ett annat bra verktyg är shellcheck (installeras med sudo apt install shellcheck), som läser igenom ett skript utan att köra det och pekar ut vanliga misstag – till exempel variabler som saknar citattecken, vilket som du sett ovan kan orsaka svårupptäckta buggar.

Uppgift 9.7: Kör bash -x på ett av dina tidigare skript och beskriv med egna ord vad som skiljer sig i utskriften jämfört med en vanlig körning. Lägg därefter till set -euo pipefail högst upp i backup.sh, och testa vad som händer om du anger en källmapp som inte finns – skiljer sig beteendet från innan?

Köra skript automatiskt

Ett skript som fungerar perfekt när du kör det själv i terminalen kan plötsligt sluta fungera när det körs via cron, som du redan sett i Fördjupande uppgift 6 – och orsaken är nästan alltid densamma. cron kör skriptet i en betydligt enklare miljö än din inloggade terminal, med ett kortare \$PATH, vilket gör att kommandon du tar för givet ibland inte hittas alls. Två vanor löser de allra flesta av de här problemen: använd alltid fullständiga sökvägar (/usr/bin/tar istället för bara tar) och skicka både vanlig utdata och felmeddelanden till en loggfil, så att du faktiskt kan se vad som händer:

```
0 3 * * * /home/elev/skript/backup.sh >>
/home/elev/skript/backup_cron.log 2>&1
```

2>&1 betyder "skicka felmeddelanden (fildeskriptor 2) dit vanlig utdata redan skickas (fildeskriptor 1)" – utan den raden hade eventuella fel försvunnit helt spårlöst, eftersom ingen sitter och tittar på skärmen klockan tre på natten när cron faktiskt kör skriptet.

Uppgift 9.8: Uppdatera din crontab-rad från Fördjupande uppgift 6 så att både vanlig utdata och felmeddelanden loggas till en fil. Testa genom att tillfälligt ändra källmappen i skriptet till något som inte finns, kör crontab-raden manuellt (kopiera kommandot och kör det direkt i terminalen) och kontrollera att felmeddelandet faktiskt hamnade i loggfilen.

Sammanfattande projekt: healthcheck.sh

Knyt ihop hela delen genom att bygga ett samlat systemhälsokontrollskript – precis den typ av verktyg många sysadmins har igång på riktiga servrar, ofta i kombination med larm som skickas ut om något går fel. Poängen är inte att börja om från noll, utan att återanvända logiken du redan skrivit i tidigare uppgifter och samla den i ett gemensamt skript, där varje kontroll får bli sin egen tydligt namngivna funktion.

Uppgift 9.9: Bygg healthcheck.sh som kör flera kontroller efter varandra, var och en som en egen funktion: kontroll_disk (återanvänd logiken från diskcheck.sh), kontroll_minne (varna om ledigt minne är under en gräns, från free), kontroll_processer (kontrollera att en namngiven tjänst/process faktiskt körs, t.ex. med pgrep) samt kontroll_natverk (återanvänd logiken från pingcheck.sh mot en lista servrar). Skriptet ska stödja -v för utförlig utskrift (via getopts), skriva en sammanfattning när alla kontroller är klara, och avsluta med exit 0 om allt är okej eller exit 1 om någon kontroll misslyckades – så att skriptet går att använda tillsammans med cron och larma vid problem.

Ett skript som avslutar med rätt exit code – 0 om allt är okej, ett annat tal om något är fel – är precis vad övervaknings- och automatiseringsverktyg förväntar sig för att kunna avgöra om de ska larma eller inte. Det är samma princip du kommer stöta på igen längre fram, oavsett om det handlar om CI/CD-pipelines eller konfigurationshanteringsverktyg som Ansible: ett verktyg som inte bara gör något, utan också tydligt talar om huruvida det lyckades.

Avslutande uppgifter – vanliga sysadmingörsmål

Här får du sex uppgifter som var och en speglar något en systemadministratör gör regelbundet i skarpt läge. Du behöver inga nya verktyg utöver de kommandon du redan mött i Del 7 (användare och grupper, processer, loggar) – här är det bara du själv som avgör lösningen. Samtliga sex uppgifter ska göras.

Uppgift 9.10 – Ny användare, automatiserat: Skriv ett skript new_user.sh som tar ett användarnamn som argument, kontrollerar att användaren inte redan finns, skapar användaren med en hemkatalog (adduser eller useradd -m), lägger till användaren i en angiven grupp, och loggar vad som gjordes – och av vem – till en loggfil.

Uppgift 9.11 – Loggstädning: Skriv ett skript log_cleanup.sh som tar en mapp och ett antal dagar som argument, och flyttar (eller tar bort) filer i mappen som är äldre än så många dagar. Tips: find /sökväg -mtime +N hittar filer som ändrades för mer än N dagar sedan.

Uppgift 9.12 – Tjänstevakt: Skriv ett skript `service_watch.sh` som tar ett tjänstenamn som argument, kontrollerar om tjänsten kör (`systemctl is-active`), och både startar om den och loggar händelsen om den visar sig ha kraschat.

Uppgift 9.13 – Diskrapport per användare: Skriv ett skript `home_usage.sh` som loopar igenom hemkatalogerna i `/home` och skriver ut hur mycket utrymme varje användare använder (du -sh), med den som använder mest överst.

Uppgift 9.14 – Lösenord som snart går ut: Skriv ett skript `password_report.sh` som loopar igenom systemets användare och listar vilka som har lösenord som går ut inom ett antal dagar du bestämmer. Tips: `chage -l användarnamn` visar när ett lösenord går ut.

Uppgift 9.15 – Portkontroll: Skriv ett skript `port_check.sh` med en array av portnummer, som kontrollerar vilka av dem som är öppna och lyssnar på servern (`ss -tuln`) och rapporterar resultatet för varje port.

Sammanfattning – checklista

Bocka av allt eftersom du känner dig säker på momenten:

- ☐ Jag kan skriva ett skript, göra det körbart med `chmod +x` och köra det.
- ☐ Jag kan använda variabler, positionsargument (`$1`, `$2`, `$#`) och `read`.
- ☐ Jag kan använda `if/elif/else` och filtertester (`-e`, `-f`, `-d`) i ett skript.
- ☐ Jag kan loopa med `for` och `while`.
- ☐ Jag förstår vad en exit code (`$?`) är och varför den är viktig.
- ☐ Jag kan skapa en array och loopa över den.
- ☐ Jag kan dela upp ett skript i funktioner och hantera enkel felhantering.
- ☐ Jag kan bygga ett skript som tar emot flaggor med `getopts`.
- ☐ Jag kan felsöka ett skript med `bash -x` och känner till `set -euo pipefail`.
- ☐ Jag vet vad jag behöver tänka på när ett skript ska köras via `cron` (sökvägar, loggning).
- ☐ Jag har byggt ett samlat skript (`healthcheck.sh`) som återanvänder flera tidigare funktioner.
- ☐ Jag har gjort samtliga sex avslutande uppgifter (9.10–9.15).

Del 10 – Dual boot: installera Linux Mint vid sidan av Windows

Hittills har du kört Linux i en virtuell maskin – praktiskt för att experimentera fritt, men med lite sämre prestanda eftersom den virtuella maskinen delar hårdvara med Windows. I den här delen installerar du istället Linux Mint direkt på datorns hårddisk,

vid sidan av ett befintligt Windows – en så kallad dual boot-installation. Vid uppstart får du välja vilket operativsystem du vill köra, och Linux får full tillgång till datorns hårdvara.

Viktigt: Den här delen ändrar på riktigt i datorns partitioner, till skillnad från övningarna i en virtuell maskin. Följer du guiden noggrant är risken låg, men säkerhetskopiera alltid viktiga filer först och läs igenom hela avsnittet en gång innan du börjar.

Vad är dual boot, och varför?

Dual boot betyder att två operativsystem – i det här fallet Windows och Linux Mint – installeras sida vid sida på samma dator, på varsin del av hårddisken. En bootmanager som heter GRUB visas när datorn startar, och låter dig välja vilket av de två systemen du vill köra just den gången. Det är en vanlig lösning bland systemadministratörer och utvecklare som vill arbeta i Linux dagligen, men fortfarande behöver Windows kvar för enstaka program eller spel som inte fungerar lika bra i Linux.

Innan du börjar

- Säkerhetskopiera viktiga filer – till en extern disk eller molntjänst.
- Se till att bärbara datorer är anslutna till ström under hela installationen.
- Kontrollera att du har minst 30–40 GB ledigt diskutrymme (mer är bättre).
- Om hårddisken är krypterad med BitLocker behöver krypteringen pausas innan du ändrar partitionerna.

Viktigt: Windows funktion Snabbstart (Fast Startup) försätter Windows i ett viloläge istället för att stänga av det helt, vilket kan låsa filsystemet och orsaka problem vid dual boot. Den måste stängas av innan du fortsätter – se steg 1 nedan.

1. Förbered Windows

Börja med att inaktivera Snabbstart: öppna Kontrollpanelen → Strömalternativ → Välj vad strömknapparna gör → klicka "Ändra inställningar som för närvarande inte är tillgängliga" → avmarkera "Aktivera snabbstart" → Spara ändringar.

Frigör sedan diskutrymme åt Linux Mint: högerklicka på Start-knappen → Diskhantering. Högerklicka på din Windows-partition (oftast C:) och välj "Krymp volym". Ange hur mycket du vill krympa den med i MB – t.ex. 40000 för ungefär 40 GB. Utrymmet visas därefter som "Ej allokerat". Rör inte det utrymmet mer – Linux Mints installationsprogram tar hand om det senare.

2. Skapa ett startbart USB-minne

- Ladda ner Linux Mint (Cinnamon-utgåvan) från linuxmint.com/download.php.
- Ladda ner och installera Rufus (rufus.ie) på en dator som redan kör Windows.
- Sätt i ett USB-minne på minst 4 GB – allt innehåll på det raderas.

- Öppna Rufus, välj USB-minnet under Device och Linux Mint-filen (.iso) under Boot selection, och klicka START. Behåll standardinställningarna om Rufus frågar.

3. Justera UEFI/BIOS-inställningar

Starta om datorn och tryck på tangenten för att komma in i UEFI/BIOS-inställningarna (vanligtvis F2, F10, F12 eller Del – står ofta i en kort text när datorn startar, eller gå efter din dators/tillverkares manual).

- Kontrollera bootordningen så att USB-minnet startas före den interna hårddisken, eller använd en engångs-bootmeny (ofta F12) för att direkt välja USB-minnet.
- Om datorn inte startar från USB-minnet alls, prova att stänga av Secure Boot – Linux Mint fungerar med Secure Boot på de flesta datorer, men enstaka tillverkare kräver att det stängs av.

Spara ändringarna (ofta F10) och låt datorn starta om.

4. Testa Linux Mint live

Datorn startar nu in i en live-session direkt från USB-minnet, utan att något ändras på hårddisken ännu. Använd tillfället till att testa att wifi, ljud och skärmutplösning fungerar som förväntat innan du fortsätter. När du är redo, dubbelklicka på ikonen "Install Linux Mint" på skrivbordet.

5. Installationsguiden

- Välj språk och tangentbordslayout (svenska fungerar bra).
- Kryssa i "Installera multimediatest" om datorn har internetuppkoppling – då fungerar t.ex. video och ljud direkt.

Det viktigaste steget är Installationstyp. Om installationsprogrammet hittar din Windows-installation visas alternativet "Installera Linux Mint vid sidan av Windows Boot Manager" – välj det. Det är både det enklaste och det säkraste alternativet: installationsprogrammet sköter partitioneringen åt dig utifrån det olokerade utrymmet du skapade i steg 1, och rör inte din befintliga Windows-partition.

6. Manuell partitionering (för den som vill ha kontroll)

Vill du bestämma partitioneringen själv kan du istället välja "Något annat". Då markerar du det olokerade utrymmet och skapar minst en ny partition: en root-partition (/) formaterad som ext4, som får ta merparten av utrymmet. Linux Mint använder numera en swap-fil istället för en egen swap-partition, så du behöver oftast inte skapa en sådan själv.

Viktigt: Väljer du manuell partitionering måste du också välja rätt "Enhet för bootloader-installation" – det ska vara hela disken (t.ex. /dev/sda), inte en enskild

partition. Väljer du fel kan Windows sluta gå att starta. Är du osäker, använd hellre det automatiska alternativet i steg 5.

7. Slutför installationen

- Välj tidszon (Stockholm).
- Ange ditt namn, ett användarnamn och ett lösenord för ditt nya Linux-konto.
- Installationsprogrammet kopierar nu filer och installerar systemet – tar vanligtvis 15–20 minuter.
- När installationen är klar startar du om och tar bort USB-minnet när du blir uppmanad till det.

8. Efter installationen: GRUB och första uppstarten

Vid nästa omstart möts du av GRUB – en enkel textmeny där du väljer mellan Linux Mint och Windows Boot Manager. Gör du inget val startar Linux Mint automatiskt efter en kort nedräkning. Välj Linux Mint och logga in med kontot du skapade.

Uppdatera systemet direkt, precis som du gjort i tidigare delar av häftet:

```
sudo apt update  
sudo apt upgrade -y
```

Öppna även Update Manager (eller Driver Manager) från menyn för att kontrollera om det finns rekommenderade drivrutiner, t.ex. för grafikkort.

Om något går fel

- Datorn startar rakt in i Windows utan att visa GRUB-menyn: gå tillbaka in i UEFI-inställningarna och kontrollera bootordningen – den kan ha återställts.
- Windows startar inte längre: detta är anledningen till att en säkerhetskopia är så viktig. Windows egna återställningsmedia (skapat på en annan dator) kan ofta reparera startsekvensen.
- Wifi hittas inte i Linux Mint: bero ofta på att ett proprietärt drivrutinspaket saknas – kontrollera Driver Manager, eller anslut tillfälligt med nätverkskabel för att kunna installera det.

Uppgift 10.1: Säkerhetskopiera dina viktiga filer, inaktivera Snabbstart och frigör minst 30 GB ledigt utrymme i Windows enligt guiden ovan.

Uppgift 10.2: Skapa ett startbart USB-minne med Linux Mint med hjälp av Rufus.

Uppgift 10.3: Installera Linux Mint vid sidan av Windows (dual boot). Starta därefter om datorn och kontrollera att GRUB-menyn visas och att du kan välja mellan båda operativsystemen.

Uppgift 10.4: Uppdatera det nyinstallerade systemet med apt, och kontrollera att wifi, ljud och skärmutlösning fungerar som förväntat.